

Week 8 : GIT - HOL

1.GIT – HOL

Git Configuration

Commands used:

```
git config --global user.name "yokessh"  
git config --global user.email "727822tuio062@skct.edu.in"
```

Output (example):

```
pidar@subalakshmiP MINGW64 ~  
$ git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=openssl  
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt  
core.autocrlf=true  
core.fscache=true  
core.symlinks=false  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
user.name=suba-2005  
user.email=22csecs.subalakshmi.p@skct.edu.in  
  
pidar@subalakshmiP MINGW64 ~  
$ git config --global user.name "suba-2005"  
bash: $'\302\226\302\226\302\226git': command not found
```

Integrating Notepad++

Steps:

1. Locate Notepad++ (e.g., C:\Program Files\Notepad++)
2. Add to PATH via:
 - Control Panel → System → Advanced system settings → Environment Variables
 - Edit "Path" under User Variables → Add Notepad++ path
3. Set as default Git editor: `git config --global core.editor "notepad++"`
4. Verify: `git config --global -e`

6. Create Local Repository

```
mkdir GitDemo  
cd GitDemo  
git init
```

Week 8 : GIT - HOL

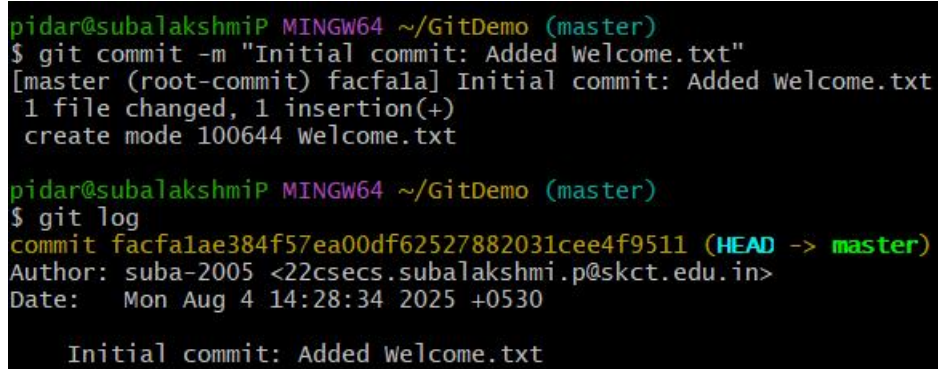
7. Add File to Repository

```
echo "Welcome to GitLab hands-on session" > welcome.txt
git add welcome.txt
git commit -m "Initial commit: Added Welcome.txt"
```

8. Verify Commit

```
git log
```

OUTPUT

A terminal window with a black background and green text. The prompt is 'pidar@subalakshmiP MINGW64 ~/GitDemo (master)'. The first command is '\$ git commit -m "Initial commit: Added Welcome.txt"', followed by the output: '[master (root-commit) facfala] Initial commit: Added Welcome.txt', '1 file changed, 1 insertion(+)', and 'create mode 100644 Welcome.txt'. The second command is '\$ git log', followed by the output: 'commit facfala384f57ea00df62527882031cee4f9511 (HEAD -> master)', 'Author: suba-2005 <22csecs.subalakshmi.p@skct.edu.in>', 'Date: Mon Aug 4 14:28:34 2025 +0530', and 'Initial commit: Added Welcome.txt'.

2. GIT – HOL

1. Create a remote GitLab repository (if not already done)

- Go to <https://gitlab.com/>
- Click **+** New Project
- Name it: GitDemo
- Visibility: **Private** or **Public** (your choice)
- Click **Create Project**

2. Copy the HTTPS clone URL

<https://gitlab.com/your-username/GitDemo.git>

3. Link your local repo to GitLab

```
git remote add origin https://gitlab.com/your-username/GitDemo.git
```

(Replace the URL with your actual GitLab repo link)

4. Push your code to GitLab

If this is your **first push**, use:

```
git push -u origin master
```

(If your default branch is main on GitLab, replace master with main)

Week 8 : GIT - HOL

OUTPUT

```
pidar@subalakshmiP MINGW64 ~
$ cd ~/GitDemo

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ echo "This is log file" > debug.log

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ mkdir log

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ echo "Another log file" > log/errors.log

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ touch .gitignore

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ notepad++ .gitignore

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ ^C

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git add .gitignore

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git commit -m "Added .gitignore to ignore .log files and log folder"
[master 20c933f] Added .gitignore to ignore .log files and log folder
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean
```

3.GIT – HOL

PART 1: Branching in Git

Step 1: Create and switch to a new branch

```
git checkout -b GitNewBranch
```

Step 2: List all branches

```
git branch -a
```

Step 3: Add a file and write content

```
echo "This is a test file in GitNewBranch" > newfile.txt
```

Week 8 : GIT - HOL

Step 4: Stage the file

```
git add newfile.txt
```

Step 5: Commit the changes

```
git commit -m "Added newfile.txt in GitNewBranch"
```

Step 6: Check status

```
git status
```

PART 2: Merging in Git

Step 1: Switch to master

```
git checkout master
```

Step 2: Show CLI differences

```
git diff master GitNewBranch
```

Step 3: Visual difference using P4Merge

```
git difftool master GitNewBranch
```

P4Merge opens and shows visual diff.

Step 4: Merge GitNewBranch into master

```
git merge GitNewBranch
```

Step 5: Visualize merge history

```
git log --oneline --graph --decorate
```

Step 6: Delete the merged branch

```
git branch -d GitNewBranch
```

PART 3: Using GitLab for Branch and Merge Requests

Create a Branch in GitLab:

1. Go to your GitLab project → Repository → Branches
2. Click on **New branch**
3. Name it GitNewBranch, and base it off main
4. Click **Create branch**

Push your local branch to GitLab:

```
git push origin GitNewBranch
```

Week 8 : GIT - HOL

OUTPUT

```
pidar@subalakshmiP MINGW64 ~
$ git checkout -b GitNewBranch
fatal: not a git repository (or any of the parent directories): .git

pidar@subalakshmiP MINGW64 ~
$ ^C

pidar@subalakshmiP MINGW64 ~
$ mkdir GitDemo
mkdir: cannot create directory 'GitDemo': File exists

pidar@subalakshmiP MINGW64 ~
$ cd GitDemo

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git init
Reinitialized existing Git repository in C:/Users/pidar/GitDemo/.git/

pidar@subalakshmiP MINGW64 ~/GitDemo (master)
$ git checkout -b GitNewBranch
Switched to a new branch 'GitNewBranch'

pidar@subalakshmiP MINGW64 ~/GitDemo (GitNewBranch)
$ echo "This is a test file in GitNewBranch" > newfile.txt

pidar@subalakshmiP MINGW64 ~/GitDemo (GitNewBranch)
$ git add newfile.txt
warning: in the working copy of 'newfile.txt', LF will be replaced by CRLF the next time Git touches

pidar@subalakshmiP MINGW64 ~/GitDemo (GitNewBranch)
$ git commit -m "Added newfile.txt in GitNewBranch"
[GitNewBranch 7234528] Added newfile.txt in GitNewBranch
1 file changed, 1 insertion(+)
create mode 100644 newfile.txt

pidar@subalakshmiP MINGW64 ~/GitDemo (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean

pidar@subalakshmiP MINGW64 ~/GitDemo (GitNewBranch)
$ |
```

4.GIT – HOL

Step 1: Initialize a Git repository (if not already done)

```
mkdir GitLabConflictDemo
cd GitLabConflictDemo
git init
```

Step 2: Create a base file and make an initial commit on master

```
echo "Initial setup" > setup.txt
git add setup.txt
git commit -m "Initial commit on master with setup.txt"
```

Step 3: Create a new branch GitWork and switch to it

```
git checkout -b GitWork
```

Week 8 : GIT - HOL

Step 4: Create `hello.xml` and add content in `GitWork`

```
echo "<message>Hello from GitWork</message>" > hello.xml
git add hello.xml
git commit -m "Added hello.xml in GitWork"
```

Step 5: Switch back to master

```
git checkout master
```

Step 6: Create a different version of `hello.xml` in master

```
echo "<message>Hello from master branch</message>" > hello.xml
git add hello.xml
git commit -m "Added hello.xml in master"
```

Step 7: View log history of both branches

```
git log --oneline --graph --decorate --all
```

Step 8: Check differences via CLI

```
git diff GitWork master
```

Step 9: View differences visually using P4Merge

```
git difftool GitWork master
```

Step 10: Merge `GitWork` into `master`

```
git merge GitWork
```

Step 11: Observe the conflict markers in `hello.xml`

```
cat hello.xml
```

Step 12: Resolve conflict using 3-way merge tool (P4Merge)

```
git mergetool
```

Step 13: Add and commit the resolved file

```
git add hello.xml
git commit -m "Resolved merge conflict in hello.xml"
```

Step 14: Add `.orig` backup file pattern to `.gitignore`

```
echo "*.orig" >> .gitignore
git add .gitignore
git commit -m "Ignore merge backup files"
```

Week 8 : GIT - HOL

Step 15: List all branches

git branch

Step 16: Delete the merged branch

git branch -d GitWork

Step 17: View final log history

git log --oneline --graph --decorate

OUTPUT

```
MINGW64:/c:/Users/pidar/GitLabConflictDemo
nothing to commit (create/copy files and use "git add" to track)

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ echo "Initial setup" > setup.txt

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git add setup.txt
warning: in the working copy of 'setup.txt', LF will be replaced by CRLF the next time Git touches it

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git commit -m "Intial commit on master with setup.txt"
[master (root-commit) 3f34938] Intial commit on master with setup.txt
1 file changed, 1 insertion(+)
create mode 100644 setup.txt

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ get checkout -b GitWork
bash: get: command not found

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (GitWork)
$ echo "<message>Hello from GitWork</message>" > hello.xml

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (GitWork)
$ git commit -m "Added hello.xml in GitWork"
[GitWork d75008e] Added hello.xml in GitWork
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (GitWork)
$ git checkout master
Switched to branch 'master'

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ echo "<message>Hello from master branch</message>" > hello.xml

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git commit -m "Added hello.xml in master"
[master ddddfc5] Added hello.xml in master
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

pidar@subalakshmiP MINGW64 ~/GitLabConflictDemo (master)
$ git log --oneline --graph --decorate --all
* ddddfc5 (HEAD -> master) Added hello.xml in master
| * d75008e (GitWork) Added hello.xml in GitWork
```

Week 8 : GIT - HOL

5. GIT – HOL

Step 1: Create a new Git project folder and initialize Git

```
mkdir GitPushDemo
cd GitPushDemo
git init
```

Step 2: Create and commit a sample file

```
echo "This is from Git-T03-HOL_002" > lab_notes.txt
git add lab_notes.txt
git commit -m "Added lab_notes.txt as part of Git-T03-HOL_002"
```

Step 3: Create a new repository on GitHub

1. Go to <https://github.com>
2. Click **New repository**
3. Enter repository name: GitPushDemo
4. Choose public or private
5. **Do not** initialize with README
6. Click **Create repository**

Step 4: Link local repo with remote GitHub repo

Replace <your-username> with your GitHub username:

```
git remote add origin https://github.com/<your-username>/GitPushDemo.git
```

Step 5: Verify if master is in clean state

```
git status
```

Step 6: List available branches

```
git branch -a
```

Step 7: Pull from remote (first time repo might not have content)

```
git pull origin master
```

Step 8: Push local commits to remote GitHub repo

```
git push -u origin master
```

Step 9: Verify the changes in the remote repo

1. Go to <https://github.com/<your-username>/GitPushDemo>
2. You should see:
 - o lab_notes.txt
 - o Your commit message: “Added lab_notes.txt as part of Git-T03-HOL_002”

Week 8 : GIT - HOL

OUTPUT

```
pidar@subalakshmiP MINGW64 ~/GitlabConflictDemo/GitPushDemo (master)
$ git remote add origin https://github.com/suba-2005/GitPushDemo.git

pidar@subalakshmiP MINGW64 ~/GitlabConflictDemo/GitPushDemo (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 281 bytes | 281.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/suba-2005/GitPushDemo.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

pidar@subalakshmiP MINGW64 ~/GitlabConflictDemo/GitPushDemo (master)
$ |
```